Computer Vision

Master Summary

Stefan Eder

Based on Lecture *Computer Vision* SS26

This document is a compact summary of the course. It aims to cover all relevant concepts and results, but long derivations, detailed proofs, and worked examples are often condensed or omitted in favour of clarity and conciseness. Use it as a study aid alongside the lecture notes and slides, not as a replacement for them.

If you spot an error or a gap, please open an issue or pull request at **github.com/stevetgit** — corrections are always welcome.

Good study materials should be shared, not gatekept. If this helped you, take it as a reason to help the next colleague you see struggling with something.

March 13, 2026

Contents

1 Introduction . . . **1**
1.1 Classical CV Task Landscape . . . 2

2 Capturing Digital Images . . . **3**
2.1 Image Formation Models . . . 3
2.1.1 Pinhole Camera and Perspective Projection . . . 3
2.1.2 Projection Models: Perspective, Orthographic, and Weak Perspective . . . 3
2.1.3 Thin Lens Model: Focal Length, Focus, and Aperture . . . 4
2.1.4 Depth of Field and f-Number . . . 5
2.2 Digital Sensors and the Imaging Pipeline . . . 5
2.2.1 Sampling and Quantization . . . 6
2.2.2 Noise Sources and SNR . . . 6
2.2.3 Color Filter Arrays and Demosaicing . . . 6
2.3 Color Imaging . . . 7
2.4 Camera Geometry and the Camera Matrix . . . 7
2.4.1 Coordinate Systems and Homogeneous Coordinates . . . 7
2.4.2 Perspective Division and Extrinsics . . . 7
2.4.3 Projection Matrix and Intrinsics . . . 8
2.5 Distortion and Calibration . . . 8
2.5.1 Radial Distortion Model . . . 8
2.5.2 Checkerboard Calibration . . . 9
2.6 Active Depth Sensors and Event Cameras . . . 9
2.6.1 Time-of-Flight and LiDAR . . . 9
2.6.2 Event Cameras (DVS) . . . 9

3 Digital Image Processing . . . **10**
3.1 Images as Discrete Signals . . . 10
3.1.1 Neighborhoods and Boundary Handling . . . 10
3.1.2 Correlation vs. Convolution . . . 10
3.2 Spatial Filtering . . . 11
3.2.1 Smoothing Filters . . . 11
3.2.2 Derivatives and Edges . . . 11
3.2.3 Second Derivatives and Unsharp Masking . . . 11
3.3 Frequency Domain . . . 12
3.3.1 Fourier Transform . . . 12
3.3.2 Sampling and Aliasing . . . 12
3.4 Multi-Scale Representations . . . 12
3.4.1 Gaussian and Laplacian Pyramids . . . 12
3.5 Inverse Problems . . . 13
3.5.1 Blur Models and Naive Inverse Filtering . . . 13
3.5.2 Regularization . . . 14
3.6 Gradient-Domain Processing . . . 14
3.6.1 Gradients as a Vector Field . . . 14

3.6.2 Poisson Reconstruction . . . 14
3.7 CNN Note: Transposed Convolution . . . 15

4 Machine Learning . . . 16
4.1 Dimensionality Reduction . . . 16
4.1.1 Principal Component Analysis . . . 16
4.1.2 Eigenfaces . . . 16
4.2 Deep Learning for Images: CNN Essentials . . . 17
4.2.1 Convolution Layer Computation . . . 17
4.2.2 Nonlinearities, Pooling, and Receptive Field . . . 18
4.2.3 Classifier Head and Dense Prediction . . . 18

5 Feature Extraction . . . 19
5.1 What Makes a Good Feature? . . . 19
5.2 Model-Based vs. Learning-Based Feature Extraction . . . 19
5.2.1 Bag of Visual Words . . . 19
5.3 Invariances . . . 20
5.4 SIFT (Scale-Invariant Feature Transform) . . . 20
5.4.1 The SIFT Pipeline . . . 20
5.5 CNN Features as Descriptors . . . 20

6 Segmentation . . . 22
6.1 What is Segmentation? . . . 22
6.2 Segmentation by Clustering . . . 22
6.2.1 K-Means Clustering . . . 23
6.3 Segmentation with Graphs . . . 23
6.3.1 Spectral Clustering . . . 23
6.3.2 Graph Cut and GrabCut . . . 23
6.4 Segmentation by Fitting . . . 24
6.4.1 Hough Transform . . . 24
6.5 Segmentation by Learning . . . 24
6.5.1 Fully Convolutional Networks . . . 24
6.5.2 U-Net and Skip Connections . . . 25

7 Optical Flow . . . 26
7.1 Definition and Use Cases . . . 26
7.2 Core Assumptions . . . 26
7.3 Optical Flow Constraint Equation . . . 27
7.3.1 Derivation . . . 27
7.3.2 Aperture Problem . . . 27
7.4 Classical Methods . . . 27
7.4.1 Lucas–Kanade (Local Window) . . . 27
7.4.2 Coarse-to-Fine for Large Motion . . . 27
7.4.3 Horn–Schunck (Global Energy) . . . 28
7.5 Optical Flow and Learning . . . 28

8 Object Detection . . . 29
8.1 Problem Setup . . . 29
8.2 Region Proposals and the R-CNN Family . . . 29
8.2.1 Selective Search and R-CNN . . . 30
8.2.2 Fast R-CNN and Faster R-CNN . . . 30

8.2.3 Anchors and Bounding-Box Regression . . . 30
8.2.4 Mask R-CNN . . . 31
8.3 Single-Shot Detectors . . . 31
8.3.1 IoU and Non-Maximum Suppression . . . 31
8.4 Evaluation . . . 32
8.5 Vision Transformers . . . 32

9 Multi-View Geometry . . . 33
9.1 Transformations (Exam Essentials) . . . 33
9.2 Recap: Camera Model Prerequisites . . . 33
9.3 Feature Matching Across Views . . . 34
9.4 Epipolar Geometry . . . 34
9.5 Essential and Fundamental Matrices . . . 34
9.6 Three-View Geometry . . . 35
9.7 Rectification and Disparity . . . 35
9.8 Stereo Depth Reconstruction . . . 36
9.9 Affine Structure from Motion . . . 36
9.10 Structure from Motion (SfM) . . . 36

10 3D Vision . . . 38
10.1 Depth from X . . . 38
10.1.1 Depth from Stereo and Multiple Views . . . 38
10.1.2 Depth from Other Cues . . . 38
10.2 Depth Maps . . . 39
10.2.1 Structured Light with Gray Codes . . . 39
10.2.2 Light Transport and Helmholtz Reciprocity . . . 39
10.2.3 Evaluating Depth Maps . . . 39
10.2.4 Scale-Depth Ambiguity and Scale-Invariant Loss . . . 39
10.3 Surface Normals . . . 40
10.4 3D Shape Representations . . . 40
10.4.1 Voxel Grids . . . 40
10.4.2 Point Clouds . . . 40
10.4.3 Meshes . . . 41
10.4.4 Implicit Surfaces . . . 41
10.5 Neural Radiance Fields (NeRF) . . . 41
10.5.1 Camera Rays and Volume Rendering . . . 41
10.5.2 NeRF Parameterization and Training . . . 42

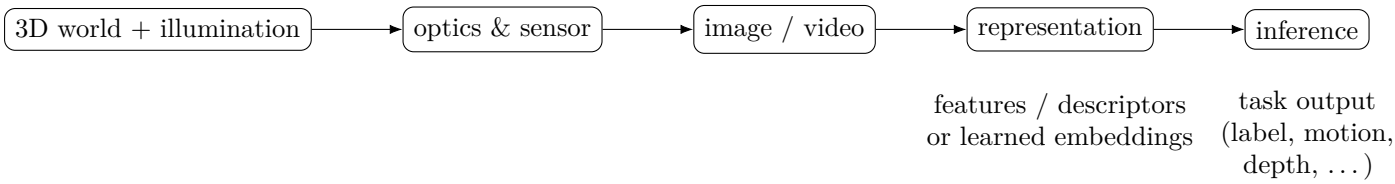
1 Introduction

Computer Vision studies how to *extract information from visual observations* (images and videos) in order to *understand* a scene and act on it. Typical outputs include labels, locations, motion estimates, geometry (depth and 3D structure), and pixel-level interpretations. The field sits at the intersection of optics, signal processing, and machine learning.

Computer Vision

Computer Vision (CV) is the discipline of designing algorithms and systems that automatically interpret visual data. Outputs include class labels, object locations, 2D motion fields, dense depth maps, and full 3D reconstructions.

The pipeline from scene to inference can be summarized as:



Intuition & Mental Model

Vision is an **inverse problem**: many different 3D scenes can produce very similar (or identical) 2D images. To do well, a system needs **assumptions** (priors) about the world.

Several fundamental difficulties make CV challenging:

- **Information loss:** projection from 3D to 2D removes depth; occlusions hide parts of the scene.
- **Appearance variation:** illumination, viewpoint, shadows, specularities, and sensor noise change pixel values.
- **Local ambiguity:** textureless regions and repeated patterns make matching and motion estimation ill-posed (aperture-style issues).
- **Scale and context:** what something *is* may depend on global context rather than local pixels.

1.1 Classical CV Task Landscape

The lecture highlights three core task families that recur throughout the course:

- **Classification / recognition:** what is in the image? (object, category, and scene labels).
- **Motion estimation and tracking:** how do pixels and objects move over time? (e.g., optical flow, tracking).
- **Scene reconstruction:** recover geometry (depth and 3D structure) from one or multiple images.

Many other tasks — segmentation, detection, pose estimation, SLAM — can be seen as combinations or refinements of the above and will appear in later chapters.

2

Capturing Digital Images

Every CV pipeline starts with image formation: how does light from a 3D scene become a discrete array of numbers? This chapter traces the path from physical optics through camera geometry to calibration and active sensors.

2.1 Image Formation Models

2.1.1 Pinhole Camera and Perspective Projection

The simplest camera model places a single aperture (the pinhole) between the scene and the image plane. All light from a scene point must pass through the pinhole, so each scene point maps to exactly one image point.

Pinhole Camera

A **pinhole camera** maps a 3D point $\boldsymbol{P} = (X, Y, Z)^\mathsf{T}$ in camera coordinates to an image point $\boldsymbol{p} = (x, y)^\mathsf{T}$ by intersecting the ray from $\boldsymbol{P}$ through a single aperture with the image plane.

Perspective Projection

For a camera coordinate system with image plane at distance f:

$$x = \frac{fX}{Z}, \qquad y = \frac{fY}{Z}.$$

There is an inherent trade-off in pinhole size: a smaller pinhole yields a sharper image (less geometric blur) but lets in less light. A larger pinhole is brighter but blurrier, because rays from one scene point spread over an area on the image plane. In practice, a *virtual image plane in front of the pinhole* is often used to avoid the "flipped image" mental picture; the mathematics stays the same.

2.1.2 Projection Models: Perspective, Orthographic, and Weak Perspective

Real cameras and CV algorithms use several approximations of perspective projection depending on the depth variation in the scene.

Projection Models

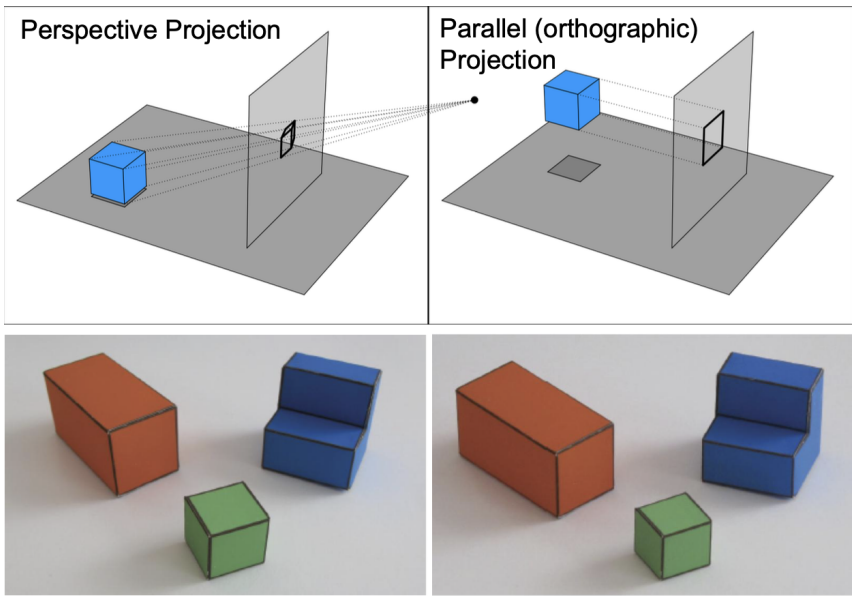
Perspective projection scales by inverse depth $1/Z$ (farther objects appear smaller).
Orthographic/Parallel projection ignores depth scaling (reasonable if depth variation is small).
Weak perspective (scaled orthographic) approximates all depths in a region by a constant reference depth Z_0 (e.g. the mean object depth), so perspective becomes a *constant-scale* projection.

Projection Equations

Let (X, Y, Z) be camera coordinates and (x, y) normalized image coordinates.

$$\begin{aligned}
\textbf{Perspective:} \quad & x = \frac{X}{Z}, \qquad y = \frac{Y}{Z}. \\
\textbf{Weak perspective:} \quad & \text{assume } Z = Z_0 + \Delta Z \text{ with } |\Delta Z| \ll Z_0 \\
& \Rightarrow \; x \approx \frac{X}{Z_0}, \qquad y \approx \frac{Y}{Z_0} \quad \text{(constant scale within the region)} \\
\textbf{Orthographic/Parallel:} \quad & x = X, \qquad y = Y \quad \text{(optionally with a constant scale factor } s\text{)}.
\end{aligned}$$

In perspective projection, 3D parallel lines can meet at a *vanishing point* in the image, and the *horizon line* is the set of vanishing points for directions parallel to the ground plane. In (weak) orthographic projection, parallel lines remain parallel — there are no vanishing points.



2.1.3 Thin Lens Model: Focal Length, Focus, and Aperture

Real cameras use lenses to collect more light than a pinhole while keeping points approximately focused. Despite using a thin lens, the mapping from 3D to 2D is still well-approximated by the linear projective (pinhole) model; more complex or thick lenses primarily require non-linear distortion models in practice.

Focal Length

The **focal length** f is the distance from the (thin) lens center to the focal point where parallel incoming rays (from infinity) converge.

Aperture

An adjustable opening controlling the *effective diameter* of the lens (how many rays are admitted).

Although a lens is used for light collection and focusing, the *geometric mapping* from 3D to 2D is still well approximated by the same perspective projection equations as the pinhole model — the lens mainly improves brightness by capturing rays from a finite solid angle.

2.1.4 Depth of Field and f-Number

Depth of Field

The **depth of field (DoF)** is the range of depths that appear acceptably sharp in the image.

f-Number

The **f-number** N is a dimensionless ratio

$$N = \frac{f}{D} \qquad \Longleftrightarrow \qquad D = \frac{f}{N},$$

where f is the focal length and D is the effective aperture diameter (entrance pupil).

- **Larger** N (smaller aperture): *less light* but *larger DoF* (more of the scene appears sharp).
- **Smaller** N (larger aperture): *more light* but *smaller DoF* (stronger background blur).

Collected light is proportional to the aperture area, so

$$\text{light} \propto D^2 \propto \frac{1}{N^2}.$$

Changing the f-number by one full "stop" changes the light by roughly a factor of 2.

f-Number Calculation

If $f = 80\,\text{mm}$ and $N = 4$, then $D = f/N = 20\,\text{mm}$.

Geometric Lens Distortion

Geometric lens distortion warps image geometry. Most important in CV: **radial distortion** (barrel vs. pincushion), especially for wide FoV lenses. Most consumer cameras need calibration before use in CV.

2.2 Digital Sensors and the Imaging Pipeline

2.2.1 Sampling and Quantization

Digital Image

A digital image is a discrete 2D function:

$$I : \{1, \ldots, W\} \times \{1, \ldots, H\} \to \{0, \ldots, 2^b - 1\},$$

where b is the **bit depth** (e.g., $b = 8$ for 256 intensity levels).

- **Sampling** (finite pixel spacing) limits spatial resolution.
- **Aliasing** occurs when scene detail exceeds the sensor sampling rate (high frequencies fold into low ones).
- **Quantization** maps continuous irradiance to discrete intensity values; larger b gives finer intensity steps.

Anti-aliasing can be helped by optical blur (lens and sensor) or explicit low-pass filtering before downsampling.

2.2.2 Noise Sources and SNR

Noise and SNR

Noise is random variation in the measured pixel value under identical conditions. The **signal-to-noise ratio (SNR)** is often more informative than absolute noise.

- **Shot noise:** photon counting statistics (Poisson distributed). For large photon counts, Poisson is well-approximated by a Gaussian.
- **Read noise:** sensor and electronics noise (approximately signal-independent).
- **Exposure trade-off:** more exposure (time or aperture) increases signal and usually improves SNR, but may cause motion blur or saturation.

Averaging multiple independent frames reduces noise variance (roughly $\propto 1/n$) and improves SNR.

Many consumer cameras apply a **non-linear response** (gamma-like) to mimic human perception. For CV it is often beneficial to **linearize** intensities before photometric computations. If starting from raw Bayer/CFA data, a common CV order is: **linearize raw CFA samples first, then demosaic** (so interpolation happens in a linear domain).

2.2.3 Color Filter Arrays and Demosaicing

Color Filter Array

Most sensors measure only **one** color per pixel using a **Color Filter Array (CFA)**. The most common CFA is the **Bayer pattern** with twice as many green samples as red or blue: **50% green, 25% red, 25% blue**.

Demosaicing (debayering) interpolates the missing channels from neighbors to obtain a full RGB image. More green samples reflect that perceived luminance is strongly influenced by green

wavelengths; it also helps spatial detail.

2.3 Color Imaging

Grayscale is often sufficient for geometry and structure (edges, corners, motion) and can simplify modeling. RGB is useful when appearance cues matter (recognition, segmentation, materials).

White balance / color constancy compensates for the illuminant so that neutral objects appear neutral and colors are comparable across lighting conditions.

2.4 Camera Geometry and the Camera Matrix

2.4.1 Coordinate Systems and Homogeneous Coordinates

We distinguish three coordinate systems:

- **World coordinates** $\boldsymbol{P}_W = (X_W, Y_W, Z_W)^\mathsf{T}$: fixed scene coordinate system.
- **Camera coordinates** $\boldsymbol{P}_C = (X_C, Y_C, Z_C)^\mathsf{T}$: origin at the camera center, Z_C points forward.
- **Image / pixel coordinates** (u, v): discrete pixel grid on the sensor.

Homogeneous Coordinates

We write points with an extra coordinate so that projections become linear.

- 2D point (x, y) becomes $\tilde{\boldsymbol{p}} = (x, y, 1)^\mathsf{T}$ (also $(\lambda x, \lambda y, \lambda)^\mathsf{T}$ for any $\lambda \neq 0$).
- 3D point (X, Y, Z) becomes $\tilde{\boldsymbol{P}} = (X, Y, Z, 1)^\mathsf{T}$ (defined only up to scale).

2.4.2 Perspective Division and Extrinsics

Perspective Division

If $\tilde{\boldsymbol{p}} = (x', y', w')^\mathsf{T}$ is a homogeneous image point, the Euclidean point is

$$(x, y) = \left(\frac{x'}{w'}, \frac{y'}{w'}\right).$$

The **extrinsics** tell us where the camera is and how it is oriented relative to the world. $\mathbf{R}$ is a 3D rotation matrix ($\mathbf{R}^\top\mathbf{R} = \mathbf{I}$, $\det(\mathbf{R}) = 1$, i.e. $SO(3)$), and $\boldsymbol{C} \in \mathbb{R}^3$ is the **camera center** in world coordinates.

World to Camera Transform

To express a world point in the camera coordinate system:

$$\boldsymbol{P}_C = \mathbf{R}(\boldsymbol{P}_W - \boldsymbol{C}) = \mathbf{R}\boldsymbol{P}_W + \boldsymbol{t}, \qquad \boldsymbol{t} = -\mathbf{R}\boldsymbol{C}.$$

2.4.3 Projection Matrix and Intrinsics

Pinhole Camera Model

$$\tilde{\boldsymbol{p}} \sim \mathbf{P}\,\tilde{\boldsymbol{P}}_W, \qquad \mathbf{P} = \mathbf{K}\,[\mathbf{R} \mid \boldsymbol{t}].$$

Intrinsic Matrix

The **intrinsics** map from normalized camera coordinates to pixel coordinates. A common intrinsic matrix form:

$$\mathbf{K} = \begin{pmatrix} f_x & s & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{pmatrix},$$

where (c_x, c_y) is the principal point, s is skew (often ≈ 0), and f_x, f_y are focal lengths in pixel units.

In many calibration contexts, **intrinsics** include $(\mathbf{K}, \text{distortion coeffs})$, i.e., lens distortion parameters (radial and tangential) are grouped with intrinsics.

2.5 Distortion and Calibration

2.5.1 Radial Distortion Model

Radial Distortion

The ideal pinhole model assumes rays travel perfectly straight. Real lenses bend light non-linearly near the edges, causing **radial distortion**.

Radial Distortion Equations

Work in **normalized** image coordinates (x, y) (after dividing by depth and removing $\mathbf{K}$). Let $r^2 = x^2 + y^2$. The distortion scale factor is:

$$s(r) = 1 + k_1 r^2 + k_2 r^4 + k_3 r^6$$

Distorted coordinates:

$$x_d = x\,s(r), \quad y_d = y\,s(r).$$

- **Barrel distortion ($k_1 < 0$):** outer points are pulled *inwards*; straight lines bow outward (typical for wide-angle/fisheye lenses).
- **Pincushion distortion ($k_1 > 0$):** outer points are pushed *outwards*; straight lines bend inward (typical for telephoto lenses).

2.5.2 Checkerboard Calibration

Camera Calibration

Estimate intrinsics $\mathbf{K}$, distortion parameters, and (for each calibration image) extrinsics $(\mathbf{R}, \boldsymbol{t})$ by observing a known pattern (e.g., checkerboard).

Reprojection Error Minimization

For every known 3D corner $\boldsymbol{P}_i$ on the checkerboard, project it into the image using current parameter estimates, compare to the detected pixel $\boldsymbol{p}_i$, and minimize:

$$\min_{\mathbf{K},\,\mathrm{dist},\,\{\mathbf{R}_j,\boldsymbol{t}_j\}} \sum_j \sum_i \left\| \boldsymbol{p}_{i,j} - \pi(\mathbf{K}, \mathrm{dist}, \mathbf{R}_j, \boldsymbol{t}_j; \boldsymbol{P}_i) \right\|^2 .$$

Calibration quality is often judged by the final RMS reprojection error (smaller is better), interpreted together with image resolution and corner detection accuracy. Accurate $\mathbf{K}$ and distortion correction are prerequisites for multi-view geometry: known relative poses $(\mathbf{R}, \boldsymbol{t})$ allow mapping rays from different cameras into one common 3D frame.

Learning-based calibration from "in-the-wild" images uses: pixel-wise undistortion mappings (e.g., U-Net predicting a dense warp), geometric cues from vanishing points/lines, or direct CNN regression of intrinsics and distortion parameters.

2.6 Active Depth Sensors and Event Cameras

2.6.1 Time-of-Flight and LiDAR

Active sensors emit structured light or laser pulses and measure the return time. **ToF distance:** $d = \frac{ct}{2}$ (round-trip time t, speed of light c). **LiDAR** is a ToF system where a laser is emitted and the return time is measured (often with scanning).

2.6.2 Event Cameras (DVS)

Standard cameras capture full frames at a fixed rate (e.g., 30 FPS), recording redundant static background and suffering from motion blur on fast objects. **Event cameras (DVS)** work like the human optic nerve: each pixel operates independently and only fires an event when its brightness changes, yielding an asynchronous stream of events $(x, y, t, \text{polarity})$. Advantages: microsecond temporal resolution, extremely low latency, no standard motion blur, and low bandwidth for static scenes.

3

Digital Image Processing

Digital image processing transforms pixel arrays to extract structure, remove noise, enhance features, or prepare data for downstream analysis. The fundamental tool is the linear filter applied via convolution.

3.1 Images as Discrete Signals

3.1.1 Neighborhoods and Boundary Handling

Discrete Image Signal

A grayscale image is a discrete 2D signal $I[n, m]$ on a pixel grid. Many operations use a local **neighborhood** (e.g., 3×3, 5×5) around each pixel to compute an output.

Typical boundary handling when the neighborhood extends outside the image:

- **Zero padding:** assume missing pixels are 0 (simple; can darken borders).
- **Replicate:** repeat boundary pixel values (often good for smoothing).
- **Reflect:** mirror the image at the boundary (often good for derivatives).
- **Circular / wrap:** treat image as periodic (matches FFT-based convolution).

Border handling changes results near image edges. For FFT-based filtering, padding the image avoids circular wrap-around artefacts.

3.1.2 Correlation vs. Convolution

Correlation and Convolution

Correlation applies a kernel h without flipping it:

$$(y = x \star h) \quad \Rightarrow \quad y[m, n] = \sum_{k,l} x[m + k, n + l]\ h[k, l].$$

Convolution flips the kernel in both directions:

$$(y = x * h) \quad \Rightarrow \quad y[m, n] = \sum_{k,l} x[m + k, n + l]\ h[-k, -l].$$

Many CV libraries implement "convolution" routines that actually compute correlation (because the kernel is learned or defined accordingly). Always check the convention.

Convolution is the prototypical **linear shift-invariant (LSI)** operation: linear combinations of inputs map to the same linear combination of outputs, and shifting the input shifts the output. Convolution is also **commutative, associative, and distributive**.

3.2 Spatial Filtering

3.2.1 Smoothing Filters

Low-Pass Filters

A **low-pass filter** removes high-frequency components (fine details/noise), producing a smoother image. The **box filter** replaces each pixel by the average of its neighborhood. The **Gaussian filter** weights central pixels higher, forming a bell curve. Key facts: convolution of Gaussians is Gaussian; the Fourier transform of a Gaussian is Gaussian; Gaussians are **separable**.

3.2.2 Derivatives and Edges

Image Gradient (Edge Detection)

Edges are strong intensity changes. Compute the gradient vector for each pixel:

1. **Horizontal change (I_x):** convolve with Sobel $K_x \begin{pmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{pmatrix}$.
2. **Vertical change (I_y):** convolve with $K_y = K_x^\top$.
3. **Edge Strength (Magnitude):** $\|\nabla I\| = \sqrt{I_x^2 + I_y^2}$.
4. **Edge Direction (Orientation):** $\theta = \operatorname{atan2}(I_y, I_x)$.

Derivatives amplify noise. Always smooth the image first (e.g., with a Gaussian filter) before computing gradients.

3.2.3 Second Derivatives and Unsharp Masking

Laplacian

The **Laplacian** ($\Delta I = I_{xx} + I_{yy}$) measures how much a pixel differs from the average of its immediate neighbors. It highlights rapid changes and acts as a **high-pass filter**.

Unsharp Masking

1. **Blur:** smooth the original image to get I_{blur}.
2. **Extract Details:** $I_{\text{details}} = I - I_{\text{blur}}$.
3. **Sharpen:** $I_{\text{sharp}} = I + \alpha\, I_{\text{details}}$.

Any kernel with a **large positive center** and **negative neighbors** (approximating Impulse − Gaussian) is a sharpening filter. The discrete Laplacian kernel (e.g., center 4, neighbors −1) performs this detail extraction in one step.

3.3 Frequency Domain

3.3.1 Fourier Transform

Fourier Decomposition

The Fourier transform decomposes an image into a sum of 2D sine/cosine waves.

- **Center (Low frequencies):** smooth gradients, global illumination, basic shapes.
- **Edges (High frequencies):** fine details, sharp edges, noise.

The transform produces complex numbers with two meaningful parts: (1) **Magnitude** — how strong is a specific frequency (the "spectrum"); (2) **Phase** — where are the structures located. Phase is far more important for preserving spatial structure and intelligibility than magnitude.

An image with **more spatial detail and sharp edges** will have a spectrum with **energy spread further out from the center** (stronger high-frequency magnitude).

Convolution Theorem

$$f = g * h \iff F = G \cdot H,$$

i.e., convolution in the spatial domain corresponds to pointwise multiplication in the Fourier domain. For large kernels, FFT-based filtering can be faster: pad image and kernel, take FFTs, multiply spectra, then inverse FFT.

3.3.2 Sampling and Aliasing

Aliasing

Aliasing occurs when sampling is too coarse: high frequencies "fold" into low frequencies. To downsample safely, first low-pass filter (anti-alias filter), then subsample.

3.4 Multi-Scale Representations

3.4.1 Gaussian and Laplacian Pyramids

Gaussian Pyramid

To build the next smaller level of the pyramid:

1. **Smooth:** apply a Gaussian filter to the current image (prevents aliasing).
2. **Downsample:** discard every second row and column.

Result: a set of increasingly small, blurred images (low-pass).

Laplacian Pyramid

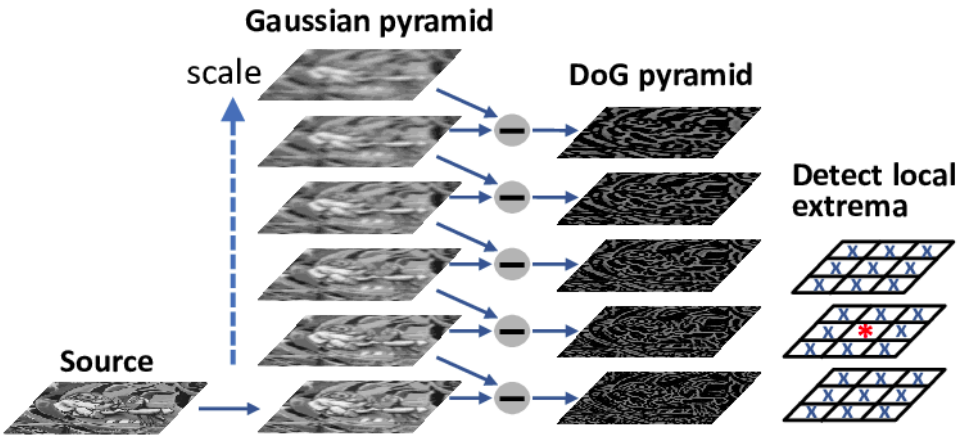
To capture what details were lost when moving to a smaller Gaussian level:

1. **Upsample:** take the smaller Gaussian level and double its size (interpolate).
2. **Subtract:** subtract the upsampled image from the original higher-resolution Gaussian level.

Result: a set of "detail images" (band-pass). The original can be perfectly reconstructed by adding details back layer by layer.

Difference of Gaussians

Difference of Gaussians (DoG): subtracting two differently blurred images of the *same size* approximates the Laplacian-of-Gaussian (LoG). This is the foundation of SIFT feature detection.



3.5 Inverse Problems

3.5.1 Blur Models and Naive Inverse Filtering

Image Degradation Model

We observe a blurry, noisy image y. We want to recover the sharp original x. Model: $y = x * h + \text{noise}$ where h is the Point Spread Function (blur kernel).

Naive Inverse Filtering

Since spatial convolution is Fourier multiplication ($Y = X \cdot H + N$), the naive approach is:

$$\hat{X} = \frac{Y}{H} = X + \frac{N}{H}.$$

Why this fails: blur kernels H destroy high frequencies, meaning $H \approx 0$ for high frequencies. Dividing the noise N by near-zero values causes N/H to **explode**, burying the image in amplified noise.

3.5.2 Regularization

Intuition & Mental Model

Regularization stabilizes inversion by trading off data fidelity and smoothness/prior assumptions. Instead of dividing by tiny H, problematic frequencies are damped.

Classical options (know the idea and when they are used):

- **Wiener filter:** frequency-dependent trade-off using noise/signal statistics.
- **Regularized filters:** add penalties (e.g., Tikhonov) to avoid unstable solutions.
- **Lucy–Richardson:** iterative deconvolution (often for Poisson noise).
- **Blind deconvolution:** estimate both x and h simultaneously (needs priors/constraints).

3.6 Gradient-Domain Processing

3.6.1 Gradients as a Vector Field

Gradient Field

The image gradient field

$$\boldsymbol{v} = (I_x, I_y)$$

can be seen as a **vector field** over pixels. If $\boldsymbol{v}$ is exactly the gradient of some image, it is **integrable** (conservative) with discrete curl ≈ 0.

After editing a gradient field (e.g., mixing gradients from different images), it typically becomes non-conservative (non-zero curl), so direct integration fails.

3.6.2 Poisson Reconstruction

Poisson Reconstruction

Given a desired vector field $\boldsymbol{v} = (v_x, v_y)$, recover an image I whose gradients match $\boldsymbol{v}$ in least-squares sense:

$$\min_I \sum_{m,n} \|\nabla I[m,n] - \boldsymbol{v}[m,n]\|^2 \quad \Longrightarrow \quad \Delta I = \nabla \cdot \boldsymbol{v},$$

a 2D Poisson equation with boundary conditions.

The Laplacian ΔI is the divergence of the gradient, so it naturally appears in the normal equations. Think of this as: *choose the image whose gradients are as close as possible to what we want.*

Common gradient-domain applications:

- **Seamless blending (Poisson blending):** paste an object while matching surrounding gradients.
- **Image fusion / reflection removal:** keep strong gradients from the desired layer, suppress others.

- **Stitching/editing:** reduce seams by enforcing consistent gradients across overlaps.
- **HDR/tone manipulation:** modify gradient magnitudes to compress dynamic range while preserving structure.

3.7 CNN Note: Transposed Convolution

Transposed Convolution (Learned Upsampling)

Often called "deconvolution", it is *not* the inverse of a convolution, but a convolution applied to an artificially inflated input.

1. **Stretch:** insert $s - 1$ zeros between every pixel (where s is the stride).
2. **Pad:** adjust the edges by subtracting padding $2p$.
3. **Convolve:** slide a standard kernel of size k over this new sparse grid.

Output size:

$$\text{out} = (\text{in} - 1) \cdot s - 2p + k.$$

4

Machine Learning

Training a model means choosing (i) a hypothesis space/model class, (ii) an objective/loss, and (iii) an optimizer. Deep learning shifts much of the feature-design burden into learning, but these three choices still matter in practice.

4.1 Dimensionality Reduction

High-dimensional image and feature vectors often lie near a lower-dimensional manifold or subspace. Reducing dimensionality can: (i) denoise, (ii) compress, (iii) make learning easier with less overfitting, and (iv) enable visualization.

Intuition & Mental Model

Geometric picture: find a low-dimensional subspace such that when you project the data onto it, the projected points are still "spread out" as much as possible.

4.1.1 Principal Component Analysis

PCA Recipe

Goal: find a new set of axes (directions) that maximize the variance/spread of the data.

1. **Center the data:** subtract the mean $\bar{\boldsymbol{x}}$ from all data points.
2. **Covariance:** compute the covariance matrix $\mathbf{C} = \frac{1}{n}\mathbf{X}^\top\mathbf{X}$.
3. **Eigen-decomposition:** find the eigenvectors and eigenvalues of $\mathbf{C}$.

Eigenvectors are the new axes (Principal Directions); **eigenvalues** give the variance along each axis. **Dimensionality reduction:** sort eigenvectors by highest eigenvalue and keep only the top-k.

4.1.2 Eigenfaces

Flatten an $N \times M$ image into a vector in $\mathbb{R}^{NM}$. PCA on a set of face images yields a mean face $\bar{\boldsymbol{x}}$ and eigenfaces $\{\boldsymbol{v}_1, \ldots, \boldsymbol{v}_k\}$ (principal directions).

Projection and Reconstruction

$$\boldsymbol{a} = \mathbf{V}_k^\top(\boldsymbol{x} - \bar{\boldsymbol{x}}), \qquad \hat{\boldsymbol{x}} = \bar{\boldsymbol{x}} + \mathbf{V}_k\boldsymbol{a},$$

where $\mathbf{V}_k = [\boldsymbol{v}_1, \ldots, \boldsymbol{v}_k]$.

Recognition via Eigenfaces

Project a test face to PCA coefficients $\boldsymbol{a}$ and do nearest-neighbor matching in coefficient space. Works if training variation matches test variation; fails under large changes in illumination, pose, or expression.

The claim that "Eigenface-based recognition is highly robust to extreme lighting/pose/expression changes" is **incorrect** — this is a common exam trap.

4.2 Deep Learning for Images: CNN Essentials

4.2.1 Convolution Layer Computation

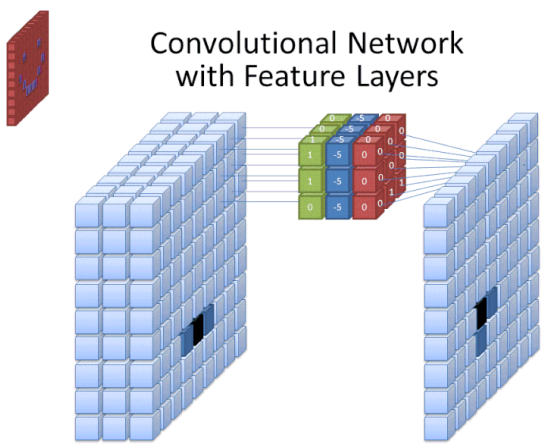
A convolution layer applies K learned filters to an input tensor $\mathbf{X} \in \mathbb{R}^{H \times W \times C}$.

How a Filter Works (3D Sliding Dot Product)

For each position in the image:

1. **Extract a 3D patch:** take a chunk of the image that matches the filter's spatial size $(k \times k)$ across *all* input channels C.
2. **Dot Product:** multiply every patch value with the corresponding filter weight and sum into a single number.
3. **Add Bias:** add a learned scalar bias b.

Sliding the same filter over the whole image gives **translation equivariance**.



CNN Dimensions and Parameters

Output Spatial Size:

$$\text{Size}_{\text{out}} = \left\lfloor \frac{\text{Size}_{\text{in}} + 2 \times \text{padding} - \text{kernel_size}}{\text{stride}} \right\rfloor + 1$$

Number of Parameters (Weights):

$$\text{Params} = (\text{kernel_width} \times \text{kernel_height} \times \text{Channels}_{\text{in}} + 1) \times \text{Filters}_{\text{out}}$$

(The +1 accounts for bias per filter.)

CNN Dimension Calculation

Size: Input 32×32, 7×7 conv, $s = 1, p = 0 \Rightarrow 26 \times 26$. Three such layers: $32 \to 26 \to 20 \to 14$.
Params: Input has 3 channels, 16 filters of size 3×3 (no bias). Each filter has $3 \times 3 \times 3 = 27$ weights. Total: $16 \times 27 = 432$.

4.2.2 Nonlinearities, Pooling, and Receptive Field

ReLU and Pooling

- **ReLU:** $f(x) = \max(0, x)$ introduces non-linearity and sparsity.
- **Pooling (e.g., max-pool):** downsamples feature maps; increases robustness to small translations, reduces computation, and increases receptive field.

- **Stride:** step size of the kernel; larger stride produces downsampling.
- **Padding:** how boundaries are handled; "same" padding keeps spatial resolution.
- **Receptive field:** region of input that influences one output activation; grows with depth and pooling.

Convolution gives *translation equivariance*, not full translation invariance. Pooling and aggregation is what increases invariance to small shifts — a common exam point.

4.2.3 Classifier Head and Dense Prediction

A typical classifier CNN ends with: spatial aggregation (flatten or global average pooling), fully connected layer(s), then class scores (logits) fed to softmax probabilities.

A convolution kernel computes a weighted sum over a local patch and produces a strong response where the patch matches the kernel. This can be used not only for whole-image classification but also for *dense* prediction — predicting the class of the center pixel from its neighborhood. This viewpoint connects CNNs naturally to semantic segmentation.

5

Feature Extraction

Early visual processing decomposes input into features (color differences, local edges/textures, motion); higher-level perception groups them into contours and regions. Good features are the foundation of matching, retrieval, and recognition pipelines.

5.1 What Makes a Good Feature?

Properties of Good Local Features

A good local feature is:

- **Repeatable:** detected at the same physical scene point across views/conditions.
- **Distinctive:** descriptor discriminates it from others (few false matches).
- **Robust:** stable under noise, blur, illumination changes, and modest viewpoint changes.
- **Efficient:** computable and matchable at scale.

5.2 Model-Based vs. Learning-Based Feature Extraction

The classic local-feature pipeline is:

$$\text{detect keypoints} \rightarrow \text{compute descriptor} \rightarrow \text{match descriptors.}$$

Used in registration, panorama stitching, Structure from Motion (SfM), and retrieval. Instead of manually designing kernels and descriptors, CNNs learn filter banks from data via backpropagation, producing task-adapted features for classification, detection, or segmentation.

5.2.1 Bag of Visual Words

BoVW Pipeline

Represent the whole image as a frequency histogram of "visual words":

1. **Extract:** find keypoints and compute local descriptors (e.g., SIFT) across all training images.
2. **Build Vocabulary:** group all descriptors into K clusters using k-means; cluster

centers are "visual words".

3. **Encode Image:** for a new image, assign each descriptor to the nearest visual word (hard assignment).
4. **Pool:** count frequencies of each word to form a single K-dimensional histogram.
5. **Classify:** feed the histogram vector into a standard classifier (e.g., SVM).

This is a useful conceptual bridge to CNNs, which *learn* filters end-to-end and also use pooling to gain robustness.

5.3 Invariances

Scale changes are ubiquitous (objects at different distances). Many hand-crafted filters are translation-equivariant detectors (Gaussian smoothing, derivatives, Laplacian). In CNNs, learned convolution kernels provide translation-equivariant feature extraction. For scale, the classic solution is **explicit multi-scale** processing (pyramids and scale space).

5.4 SIFT (Scale-Invariant Feature Transform)

5.4.1 The SIFT Pipeline

1. **Scale space (DoG):** build Gaussian pyramid and compute Difference-of-Gaussians to approximate LoG.
2. **Keypoint detection:** find local extrema of DoG in 3D neighborhood (space + scale).
3. **Filtering:** reject low-contrast (noise) and unstable (edge) points.
4. **Orientation assignment:** find dominant gradient orientation peak to normalize rotation.
5. **Descriptor construction:** take patch aligned to orientation, split into 4×4 tiles. Build 8-bin gradient histogram per tile $\Rightarrow$ **128**-D descriptor. Normalize for illumination.

SIFT Invariances and Limitations

SIFT is robust to **(i)** translation (locality), **(ii)** scale (scale space), **(iii)** rotation (orientation assignment), and moderately to illumination (normalization). It is *not* fully invariant to large viewpoint changes or heavy blur/occlusion.

Match by nearest neighbor (Euclidean distance). The ratio test (best vs. second-best distance) helps reject ambiguous matches. Failure modes: repetitive textures, large viewpoint changes, significant appearance changes.

5.5 CNN Features as Descriptors

Intermediate activations of a CNN serve as features at different levels of abstraction:

- Early layers: edges and textures (low-level). Exam fact: here you expect to find learned filter kernels that approximate Gaussians.
- Middle layers: parts and patterns (mid-level).

- Deep layers: semantic concepts (high-level).

Architecture milestones:

- **AlexNet (2012):** established modern deep CNNs for large-scale classification.
- **VGG (2014):** deeper stacks of small (3×3) convolutions.
- **GoogLeNet/Inception (2015):** multi-branch modules for efficiency.
- **ResNet (2016):** residual connections enable very deep networks.

Top-5 Error Rate

A prediction is counted as correct if the true label is among the top-5 predicted classes (so *error* means it is not in the top 5). Exam fact: ResNet can achieve a better top-5 error rate than humans.

6

Segmentation

Segmentation partitions an image into groups of pixels that belong together, assigning a label to every pixel rather than a single label for the whole image. It is fundamental to image editing, autonomous driving, and medical imaging.

6.1 What is Segmentation?

Segmentation

Segmentation = partition an image into **groups of pixels that belong together**. The output is a label per pixel (a mask).

- **Semantic segmentation:** assigns a *class label* to every pixel (all "car" pixels share the same label).
- **Instance segmentation:** separates *individual object instances* (car #1 vs. car #2).
- **Panoptic segmentation:** combines both: semantic classes *and* instance IDs where applicable.

Classification has no spatial extent; detection outputs boxes; segmentation outputs pixel masks. Do not confuse *semantic segmentation* (pixel-wise class labels) with *instance segmentation* (separate object IDs) — a common exam pitfall.

Segmentation is hard because boundaries may be weak or blurred, objects can have similar color/texture, and occlusions hide parts. Dense pixel labels are also expensive to annotate.

6.2 Segmentation by Clustering

Pixels can be modeled as points in feature space and clustered. Typical feature choices: color only ($\boldsymbol{x}_i = (R_i, G_i, B_i) \in \mathbb{R}^3$) or color + location ($\boldsymbol{x}_i = (R_i, G_i, B_i, u_i, v_i) \in \mathbb{R}^5$, which encourages spatial coherence). Adding (u, v) helps avoid grouping distant pixels that merely share color; normalize color vs. position so neither dominates distances.

6.2.1 K-Means Clustering

K-Means

k**-means** partitions $\{\boldsymbol{x}_i\}_{i=1}^N$ into k clusters by minimizing within-cluster squared distances.

K-Means Objective

$$\min_{\{\mathcal{C}_1,\ldots,\mathcal{C}_k\}} \sum_{c=1}^{k} \sum_{i\in\mathcal{C}_c} \|\boldsymbol{x}_i - \boldsymbol{\mu}_c\|^2, \qquad \boldsymbol{\mu}_c = \frac{1}{|\mathcal{C}_c|} \sum_{i\in\mathcal{C}_c} \boldsymbol{x}_i.$$

Algorithm: (1) initialize k cluster centers; (2) assign each point to nearest center; (3) recompute centers as cluster means; (4) repeat until convergence.

Typical failure modes: must choose k in advance; sensitive to initialization (local minima); not boundary-aware (can "bleed" across edges).

6.3 Segmentation with Graphs

Graph-Based Segmentation

Build a graph $G = (V, E)$ where nodes are pixels (or superpixels) and edges connect neighbors with affinity weights encoding similarity. Affinity weight a_{ij} is large if pixels are similar in color and close in space.

6.3.1 Spectral Clustering

Spectral Clustering

Instead of clustering in coordinate space, cluster based on graph connectivity:

1. **Affinity Matrix (A):** A_{ij} is high if pixel i and j are similar.
2. **Goal:** find a group assignment vector $\boldsymbol{w}$ maximizing connections within the group: $\max(\boldsymbol{w}^\top \mathbf{A}\boldsymbol{w})$.
3. **Solution:** maximizing this quadratic form (normalized $\boldsymbol{w}$) is solved by the **eigenvectors** of $\mathbf{A}$ (or the Graph Laplacian) corresponding to the largest eigenvalues.

When iteratively extracting one cluster per round and stopping when the largest eigenvalue drops below a threshold: $\#\text{clusters} = \#\{\text{rounds with } \lambda_{\max} \geq \text{threshold}\}$.

6.3.2 Graph Cut and GrabCut

Graph Cut Energy Minimization

Assign a label y_i (foreground or background) to each pixel to minimize:

$$E(\boldsymbol{y}) = \underbrace{\sum_i D_i(y_i)}_{\text{Data term}} + \lambda \underbrace{\sum_{(i,j)\in E} V_{ij}(y_i, y_j)}_{\text{Smoothness term}}$$

- **Data term:** cost of assigning a pixel to a label (does it look like foreground or background?).
- **Smoothness term:** penalizes different labels for neighboring pixels unless there is a strong natural edge.

The global optimum is found efficiently via **min-cut/max-flow** algorithms.

GrabCut is an interactive tool using Graph Cuts: the user draws a bounding box; the algorithm builds Gaussian Mixture Models (GMMs) for foreground/background, runs min-cut, updates the GMMs, and repeats iteratively.

6.4 Segmentation by Fitting

6.4.1 Hough Transform

If segments correspond to known parametric shapes (lines, circles, etc.), detect them by searching in the **parameter space** of that shape family.

Hough Transform

A voting scheme: edge pixels vote for shape parameters; peaks in an accumulator array indicate shapes.

Line Parameterization

$$\rho = x\cos\theta + y\sin\theta.$$

Each edge pixel (x, y) votes for all (ρ, θ) pairs consistent with it.

Procedure: (1) run edge detector; (2) each edge pixel votes in accumulator $H(\rho, \theta)$; (3) find peaks above threshold; (4) convert peak parameters back to image lines.

The Hough space dimension equals the number of free shape parameters: 2D line: 2 parameters (ρ, θ); 2D circle: 3 parameters (x_0, y_0, r); 2D ellipse: **5 parameters** (x_0, y_0, a, b, θ); 3D ellipsoid: typically **9 parameters**. More parameters means a larger accumulator and higher computational cost.

6.5 Segmentation by Learning

6.5.1 Fully Convolutional Networks

Learning-Based Semantic Segmentation

Uses **paired training data**: for each training image, every pixel has a class label. At test time, the model predicts a label for every pixel.

FCN Score-Tensor View

$$\underbrace{3 \times H \times W}_{\text{input}} \xrightarrow{\text{encoder (down)}} \underbrace{C \times H' \times W'}_{\text{scores}} \xrightarrow{\text{decoder (up)}} \underbrace{C \times H \times W}_{\text{scores}} \xrightarrow{\arg\max} \underbrace{H \times W}_{\text{labels}}.$$

Classification CNNs downsample aggressively to gain receptive field and efficiency, but segmentation needs **dense output at input resolution**. Two strategies: no downsampling (expensive) or encoder–decoder (downsample then upsample).

6.5.2 U-Net and Skip Connections

U-Net

Plain encoder–decoder (autoencoder) can produce blurry or imprecise boundaries because fine spatial detail is lost. **U-Net** improves this via **skip connections** that pass high-resolution encoder features directly to the decoder.

Skip connections let the decoder combine: **(i)** global semantic understanding (deep, low-resolution features) with **(ii)** local detail (shallow, high-resolution features), yielding sharper masks.

A **1×1 convolution** performs a learned linear map $\mathbb{R}^D \to \mathbb{R}^C$ per pixel independently. Uses: project feature depth D to number of classes C (per-pixel score maps), or bottlenecking for efficiency.

7

Optical Flow

Optical flow captures apparent motion in image sequences. It is the foundation of video compression, tracking, action recognition, and autonomous driving perception.

7.1 Definition and Use Cases

Optical Flow

Optical flow estimates the apparent **2D motion field** between two consecutive frames: each pixel (or region) is assigned a displacement vector (u, v) from time t to $t + 1$.

- **Sparse flow:** vectors only at selected points/patches (fast; relies on good features).
- **Dense flow:** vector for every pixel (useful for motion segmentation, tracking, video understanding).

Common use cases: video compression (motion-compensated prediction), tracking and action recognition, robotics and autonomous driving, and medical/scientific imaging (deformation fields, registration). Flow fields are often visualized with a **color wheel**: direction maps to hue, magnitude to saturation/brightness.

Stereo Connection

For a static rectified stereo-camera system and a static scene, the dense optical flow field correlates directly to a disparity map.

7.2 Core Assumptions

Three assumptions underlie classical optical flow methods:

- **Brightness constancy** (photometric consistency).
- **Small motion** (enables Taylor linearization), handled for larger motion via pyramids/warping.
- **Spatial coherence / locally constant flow** in a small window.

Brightness Constancy

$$I(x, y, t) \approx I(x + u,\ y + v,\ t + 1).$$

Brightness constancy breaks under illumination changes, specularities, motion blur, and occlusions. Smoothness assumptions can oversmooth across motion boundaries.

7.3 Optical Flow Constraint Equation

7.3.1 Derivation

Starting from $I(x+u, y+v, t+1) \approx I(x, y, t)$ and applying a first-order Taylor expansion:

Optical Flow Constraint Equation (OFCE)

$$I_x\, u + I_y\, v + I_t = 0,$$

where I_x, I_y are spatial derivatives and I_t is the temporal derivative.

7.3.2 Aperture Problem

Aperture Problem

The OFCE provides *one equation* for two unknowns (u, v) at each pixel. Along an edge, only motion *normal to the edge* is constrained; tangential motion is ambiguous.

7.4 Classical Methods

7.4.1 Lucas–Kanade (Local Window)

Lucas-Kanade Solution

1. **Assumption:** assume flow (u, v) is identical for all pixels in a small window (e.g., 3×3).
2. **Overdetermined System:** 9 equations, 2 unknowns.
3. **Least Squares:** solve for the best-fitting (u, v).

Crucial Condition: this only works if the window has gradients in at least two different directions (i.e., it must be a **corner**, not a flat region or a single straight edge).

7.4.2 Coarse-to-Fine for Large Motion

If motion exceeds approximately 1 pixel, the linearization breaks. The standard fix is **coarse-to-fine estimation**:

1. Build Gaussian pyramids of both frames.
2. Estimate flow at the coarsest level.
3. **Upsample flow**, **warp** the second image toward the first, and refine at the next finer level.
4. Repeat until full resolution.

7.4.3 Horn–Schunck (Global Energy)

Horn–Schunck Global Energy

Compute flow for *every* pixel simultaneously by minimizing:

$$E = E_{\text{data}} + \lambda E_{\text{smooth}}$$

- **Data term (E_{data}):** penalizes deviations from brightness constancy (the OFCE).
- **Smoothness term (E_{smooth}):** penalizes differences between neighboring flow vectors.

The parameter λ controls the trade-off. A larger λ forces a smoother flow field but risks washing out sharp motion boundaries where two objects move past each other.

7.5 Optical Flow and Learning

Learning-Based Flow

Learning-based flow models treat optical flow as a supervised prediction task: input = two frames, output = dense flow field (often via an encoder–decoder/U-Net-like design).

Deep methods learn a **direct mapping** from an image pair to a dense flow field. They do **not** require explicit hand-computed image gradients at inference time.

FlowNet variants from the lecture:

- **FlowNetS (Simple):** stack two RGB images into one input tensor and predict flow end-to-end.
- **FlowNetCorr (Correlation):** compute feature maps per image, then use a **correlation layer** to explicitly match features before decoding to flow.

Exam Trap

Typical failure cases for both classical and learned flow: occlusions (no correspondence), large displacements without enough pyramid/context, illumination changes (brightness constancy violated), and motion boundaries (smoothness vs. sharp edges).

8

Object Detection

Object detection must answer not just *what* is in an image but *where* it is. Detection is harder than classification because the output size is variable (unknown number of objects) and the model must jointly handle **localization** and **classification**.

8.1 Problem Setup

- **Image classification:** output a single (or multi-) label for the whole image.
- **Single-object detection:** predict class + one bounding box for the dominant object.
- **Multi-object detection:** predict a *set* of objects, each with class + bounding box (optionally a mask).

Bounding Box

A **bounding box** is typically parameterized as

$$(x, y, w, h) \quad \text{or} \quad (x_{\min}, y_{\min}, x_{\max}, y_{\max}),$$

plus a **confidence score** and a **class label**.

Multi-Task Detection Loss

$$\mathcal{L} = \mathcal{L}_{\text{cls}} + \lambda\, \mathcal{L}_{\text{box}} \ (+\ \mathcal{L}_{\text{mask}} \text{ for instance segmentation}),$$

e.g. cross-entropy for classes and a regression loss for boxes.

Localization is treated as a *regression problem* (predicting box coordinates), not classification.

8.2 Region Proposals and the R-CNN Family

8.2.1 Selective Search and R-CNN

Region Proposals

Region proposals are candidate image regions likely to contain an object. **Selective Search** generates proposals by hierarchical grouping of superpixels using similarity cues (color, texture, size, fill).

R-CNN: generate region proposals, then for each proposal warp/crop → CNN → classify and refine box. Conceptually simple but inefficient: it runs a CNN *many times* per image (once per proposal).

8.2.2 Fast R-CNN and Faster R-CNN

Fast R-CNN

Run a CNN *once* on the whole image to get a feature map, then extract fixed-size features per proposal via **RoI pooling** (or RoIAlign in later variants). Shared convolutional backbone makes it much faster than R-CNN, but still relies on external proposals.

Region Proposal Network (RPN)

Faster R-CNN replaces external proposals with an RPN that learns proposals directly from the shared feature map. At each spatial location it predicts: **(i)** an objectness score and **(ii)** box transforms for multiple anchors.

8.2.3 Anchors and Bounding-Box Regression

Anchors

Anchors are predefined reference boxes (multiple scales/aspect ratios) centered at each feature-map location. The network predicts offsets that transform an anchor into the final box.

Anchor Offset Parameterization

For anchor (x_a, y_a, w_a, h_a) and target (x, y, w, h):

$$t_x = \frac{x - x_a}{w_a}, \quad t_y = \frac{y - y_a}{h_a}, \quad t_w = \log \frac{w}{w_a}, \quad t_h = \log \frac{h}{h_a}.$$

The model predicts (t_x, t_y, t_w, t_h).

More anchors gives better coverage but more class imbalance (many negatives).

8.2.4 Mask R-CNN

Mask R-CNN

Extends Faster R-CNN with a parallel **mask head** that predicts a binary mask for each detected RoI, yielding **instance segmentation**: a mask per object instance. Detection head: class + bounding box. Mask head: per-RoI pixelwise mask (e.g. 28×28) for the predicted class.

8.3 Single-Shot Detectors

Single-shot means: one forward pass produces class scores and boxes densely over the image with no explicit proposal stage.

- **YOLO:** grid-based prediction; very fast, historically struggles with small objects (varies by version).
- **SSD:** multi-scale feature maps + default boxes (anchors) at different resolutions.
- **RetinaNet:** single-shot with **focal loss** to address extreme foreground/background imbalance.

YOLO

Splits the image into an $S \times S$ grid. The **cell containing the object center** is responsible for predicting it. Per cell it predicts B boxes and class scores.

$$Y = [p_c,\ b_x,\ b_y,\ b_w,\ b_h,\ c_1, \ldots, c_C],$$

where p_c is objectness/confidence, (b_x, b_y, b_w, b_h) are box parameters, and c_k are class scores. The full output tensor for YOLOv1-style models is $S \times S \times (5B + C)$.

8.3.1 IoU and Non-Maximum Suppression

Intersection over Union

$$\mathrm{IoU}(A, B) = \frac{\mathrm{area}(A \cap B)}{\mathrm{area}(A \cup B)} \in [0, 1].$$

Non-Maximum Suppression (NMS)

Reduces duplicate detections: keep the highest-confidence box and suppress other boxes with IoU above a threshold, then repeat.

1. Sort predicted boxes by confidence (descending).
2. Take the top box; remove all remaining boxes with IoU $> \tau$ to it.
3. Repeat until no boxes remain.

IoU is used both for *training/label assignment* (which predictions match a GT box) and for *post-processing NMS* to remove duplicates — these are two separate uses.

Dense detectors evaluate many candidate boxes, most of which are background. Standard cross-entropy can be dominated by easy negatives. **Focal loss** down-weights easy examples so the model focuses on hard, informative mistakes.

8.4 Evaluation

Detection Evaluation Basics

Standard precision/recall metrics are used, but a True Positive (TP) requires:

- **TP:** correct class AND bounding box overlaps Ground Truth by **IoU > threshold** (e.g., 0.5).
- **FP:** IoU too low, wrong class, or a duplicate.
- **FN:** an actual object was missed.

Varying the confidence threshold yields a **Precision–Recall curve**. A better detector has a curve closer to the top-right corner (high precision at high recall).

AP and mAP

Average Precision (AP) is the area under the Precision–Recall curve for a class. **mAP** is the mean AP over all classes (and sometimes over multiple IoU thresholds).

8.5 Vision Transformers

CNNs have a strong **locality** and **translation-equivariance** bias. Transformers model **global interactions** via attention, capturing long-range dependencies more directly. Historically, transformers tend to be more data-hungry than CNNs unless pretrained at scale.

Vision Transformer (ViT)

Split an image into fixed-size patches, embed each patch as a token, add positional encodings, then apply a standard transformer encoder. A special **class token** can aggregate information for classification.

9

Multi-View Geometry

A single image pixel corresponds to a *ray* in 3D — infinitely many 3D points are consistent with it. With two or more views, rays from different camera centers intersect (approximately), making depth recoverable. This chapter develops the geometry underlying stereo, feature matching, and Structure from Motion.

9.1 Transformations (Exam Essentials)

Key Transforms

Euclidean (rigid) transform: rotation + translation; preserves distances and angles.
Affine transform: Euclidean + general linear transform (scaling, shear) + translation; preserves parallel lines, not angles/lengths.

9.2 Recap: Camera Model Prerequisites

Pinhole Camera Model

$$\tilde{\boldsymbol{p}} \sim \mathbf{P}\,\tilde{\boldsymbol{P}}_W, \qquad \mathbf{P} = \mathbf{K}[\mathbf{R} \mid \boldsymbol{t}].$$

A 3×4 camera matrix has 12 entries but is defined up to scale ($\Rightarrow$ 11 DOF). Each 3D–2D correspondence gives 2 independent equations, so at least **6** correspondences are needed (DLT).

Normalized Image Coordinates

$$\tilde{\boldsymbol{x}} \sim \mathbf{K}^{-1}\tilde{\boldsymbol{p}}$$

removes the effect of intrinsics, placing points on a normalized (focal length = 1) image plane.

Epipolar geometry is derived for the ideal pinhole model. In practice, **undistort** images (using calibration) before estimating correspondences, F/E, disparity, etc.

9.3 Feature Matching Across Views

Correspondences

A **correspondence** is a pair of image points $(\boldsymbol{p}, \boldsymbol{p}')$ from the same 3D scene point. Real matching produces **outliers** due to repeated texture, occlusions, specularities, and noise.

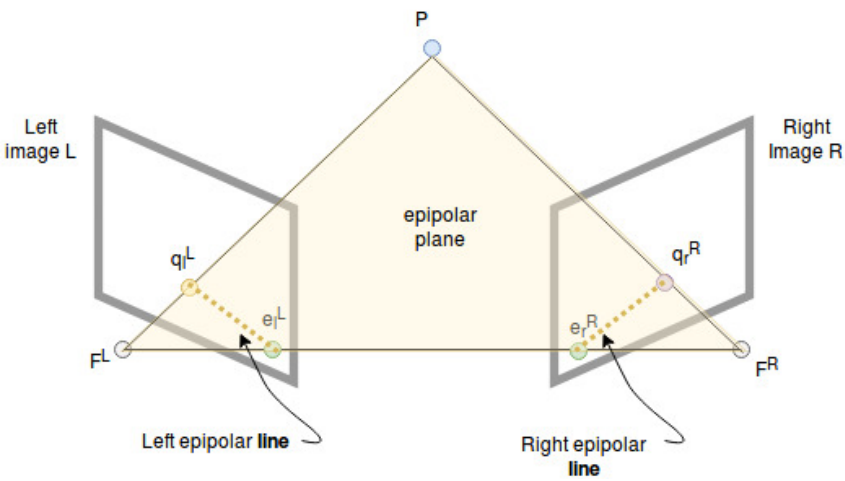
Use distinctive descriptors (e.g. SIFT) with nearest-neighbor matching, ratio test heuristics, and robust estimation (RANSAC) to handle outliers. Good sparse matches enable: estimating epipolar geometry (F/E), rectification, triangulation, and SfM pipelines.

9.4 Epipolar Geometry

Epipolar Geometry

Imagine a triangle formed by Camera A, Camera B, and the 3D object point. This triangle defines the **epipolar plane**.

- **Baseline:** the physical line connecting the two camera centers.
- **Epipoles:** what Camera A sees if it looks directly into the lens of Camera B (and vice versa).
- **Epipolar line:** the intersection of the epipolar plane with the camera's image sensor.



The epipolar constraint means that to find a pixel's match in Image B, you only need to search along a **specific epipolar line** (a fast 1D search) rather than the entire image (a 2D search).

9.5 Essential and Fundamental Matrices

E and F Relationship

$$E = K_2^\top F K_1, \qquad F = K_2^{-\top} E K_1^{-1}.$$

Use F with **pixel** coordinates and E with **normalized** coordinates. One point x in image 1 induces exactly **one** epipolar line $l_2 = Fx$ in image 2.

Epipolar Constraints

Calibrated (normalized coordinates):

$$\tilde{\boldsymbol{x}}'^{\mathsf{T}}\,\mathbf{E}\,\tilde{\boldsymbol{x}} = 0.$$

Uncalibrated (pixel coordinates):

$$\tilde{\boldsymbol{p}}'^{\mathsf{T}}\,\mathbf{F}\,\tilde{\boldsymbol{p}} = 0.$$

In the calibrated case, $\mathbf{E}$ encodes the **relative pose** (up to scale) between the cameras.

Given $\mathbf{F}$, the epipolar line in image 2 for point $\tilde{\boldsymbol{p}}$ in image 1 is $\ell' = \mathbf{F}\,\tilde{\boldsymbol{p}}$; analogously, $\ell = \mathbf{F}^{\mathsf{T}}\tilde{\boldsymbol{p}}'$ gives the line in image 1.

Estimation requires at least 8 correspondences (the classic **8-point algorithm**); more is better. Use **RANSAC** to cope with outliers: repeatedly fit $\mathbf{F}/\mathbf{E}$ on minimal samples and keep the model with the most inliers. Without robust estimation, a few bad matches can completely break estimation.

9.6 Three-View Geometry

A trifocal tensor can be represented as a $3 \times 3 \times 3$ array with **27 coefficients** (it satisfies internal constraints; its DOF is smaller than 27).

9.7 Rectification and Disparity

Rectification

Rectification warps the two images so that corresponding epipolar lines become **horizontal scanlines**. Then correspondence search becomes a 1D search along image rows.

After rectification, a correspondence typically satisfies $y \approx y'$ and the remaining unknown is the horizontal shift. **Disparity** is defined as $d = x_L - x_R$ (or opposite sign depending on convention). Dense stereo estimates a disparity $d(x, y)$ for many/all pixels, producing a **disparity map**.

Learning-based stereo uses **cost volumes** built by combining left/right features across a range of disparities d, yielding a volume of size $H \times W \times D$. A **3D CNN** then regularizes matching costs across (x, y, d) and regresses the final disparity.

The **ordering constraint** (rectified stereo): along a scanline, the left-to-right order of corresponding points is preserved for opaque non-occluded surfaces; violated by occlusions and repeated structures.

9.8 Stereo Depth Reconstruction

The Classic Stereo Formula

$$Z = \frac{f \times B}{d}$$

where Z is depth, f is focal length, B is baseline (camera separation), and d is disparity.

- **To see further:** need larger baseline B or larger focal length f.
- **Far-distance problem:** for distant objects $d \approx 0$; since $Z \propto 1/d$, a tiny 1-pixel matching error causes massive depth error.

Sparse vs. Dense Stereo

Sparse stereo: compute depth only at matched keypoints (robust, low density). **Dense stereo:** estimate depth for (almost) every pixel using matching costs and regularization.

Dense stereo struggles in textureless regions, repetitive patterns, and at occlusion boundaries.

9.9 Affine Structure from Motion

Affine shape: reconstruction is defined only **up to an affine transformation**. Two views of 4 non-coplanar points suffice to constrain affine shape; two views of 3 points do not.

A general affine camera is a 2×4 matrix (8 parameters per view). With m views and n points, unknown coefficients $\approx 8m+3n+9$, where the extra $+9$ reflects the inherent 3×3 affine ambiguity in factorization-based affine SfM.

9.10 Structure from Motion (SfM)

SfM Pipeline

SfM generalizes stereo to **multiple** images: jointly estimate **camera poses** and **3D points**.

detect/match features $\rightarrow$ estimate relative motion $\rightarrow$ triangulate $\rightarrow$ refine (bundle adjustment).

- Start from an initial image pair with good baseline.
- Incrementally add images: match to existing points, estimate new camera pose, triangulate new points.
- Refine by minimizing reprojection error (bundle adjustment).

SfM Equation Counting (m views, n points)

Each 3D point gives 2 equations per view ($2mn$ equations total).

Simple coefficient counting: camera = 12 unknowns, point = 3 unknowns. Solvable if: $2mn \geq 12m + 3n$.

Projective DOF counting: camera has 11 DOF, point has 3 DOF, coordinate system

has 15-DOF ambiguity. Solvable if: $2mn \geq 11m + 3n - 15$.

- **2 Views ($m = 2$):** needs $n > 7 \Rightarrow$ minimum **8 correspondences**.
- **3 Views / 3 Points:** $18 \geq 27 \Rightarrow$ **False** (not solvable).

SfM typically outputs camera poses for each image and a **sparse** 3D point cloud from feature tracks. Absolute **scale** is ambiguous without additional information (known baseline, IMU, GPS, etc.).

Bad matches lead to wrong $\mathbf{F}/\mathbf{E}$, wrong rectification/triangulation, and broken reconstruction. Robust matching and proper calibration/undistortion are essential.

10

3D Vision

The previous chapters interpreted images as 2D signals (classification, segmentation, detection) and recovered 3D structure from multiple views (stereo, SfM). This chapter focuses on how depth and shape can be *represented*, *predicted* (often learning-based), and *rendered* (NeRF).

10.1 Depth from X

10.1.1 Depth from Stereo and Multiple Views

Depth from Stereo

Infer depth from **parallax** (viewpoint change). Matching corresponding points constrains depth; larger baseline generally increases depth sensitivity.

- **Stereo:** two calibrated views ⇒ disparity ⇒ depth (dense or sparse).
- **Multi-view / SfM:** many views ⇒ estimate camera poses + sparse 3D points (then densify).

Classical stereo relies on: feature extraction, matching, geometry, and calibration. Classical matching is brittle: pixel appearance depends on reflectance, shape, and shading, but we only want shape, making correspondence (and thus depth) ill-posed in practice.

10.1.2 Depth from Other Cues

Depth from X

Depth estimation using cues other than multi-view parallax, e.g. **shading** or **defocus** (focus blur). Each cue requires additional assumptions about image formation.

Depth from defocus (DfD) requires **texture/high-frequency content**; in **uniform regions** blur is ambiguous. CNN features (including early/mid conv layers) can correlate with focus/blur primarily in **non-uniform (textured)** regions.

10.2 Depth Maps

Depth Map

A **depth map** stores one scalar per pixel: the distance from the camera to the observed surface point along that pixel's viewing ray. Together with RGB it forms an **RGB-D** image (often called "2.5D").

Depth maps are view-dependent: they represent only what is visible from the camera. Depth can be captured directly by structured light or time-of-flight devices (e.g., Microsoft Kinect).

10.2.1 Structured Light with Gray Codes

Gray Code Properties

Neighboring codewords differ by **Hamming distance 1** (only one bit changes), reducing decoding errors at boundaries. For resolution $W \times H$, you need $\lceil \log_2 W \rceil$ patterns for columns and $\lceil \log_2 H \rceil$ for rows. Example: 1024×1024 needs $10 + 10 = \mathbf{20}$ images.

10.2.2 Light Transport and Helmholtz Reciprocity

Model light transport as a linear map: camera image $c = T\,p$ (projector pattern p, transport matrix T). **Helmholtz reciprocity** implies a swap of emitter/sensor corresponds to the **transpose** mapping ($T^\top$). Thus **dual photography**: $p = T^\top c$. If c is uniform white, then p visualizes the scene as seen from the projector viewpoint under uniform camera-side illumination.

10.2.3 Evaluating Depth Maps

Two common viewpoints:

- **Per-pixel errors** between predicted and measured depth (after masking invalid pixels).
- **Log-depth errors** to reduce sensitivity to absolute scale and emphasize relative accuracy.

Per-Pixel Log-Depth Loss

Given ground truth y_i and prediction y_i^* at pixel i:

$$D_{L2}(y, y^*) = \frac{1}{n} \sum_{i=1}^{n} (\log y_i - \log y_i^*)^2 \,.$$

10.2.4 Scale-Depth Ambiguity and Scale-Invariant Loss

From a single image, **absolute scale** is fundamentally ambiguous without additional cues or priors — a small close object can produce the same image as a large far object.

Scale-Invariant Loss

$$D_{SI}(y, y^*) = \frac{1}{n} \sum_{i=1}^{n} (\log y_i - \log y_i^* + \alpha(y, y^*))^2 \,, \qquad \alpha(y, y^*) = \frac{1}{n} \sum_{j=1}^{n} \left(\log y_j - \log y_j^*\right) .$$

α compensates a global scale offset in log-space (a global multiplicative depth scaling).

Monocular Depth Learning

Train a CNN that maps an RGB image ($3 \times H \times W$) to a depth map ($1 \times H \times W$) using paired training data (RGB, measured depth) and a per-pixel loss. Depth supervision can come from RGB-D sensors or from multi-view reconstruction pipelines.

10.3 Surface Normals

Surface Normal Map

A **surface normal map** stores a 3D unit vector per pixel ($3 \times H \times W$) describing local surface orientation (perpendicular to the tangent plane).

Depth and normals are tightly related: depth $\rightarrow$ normals via spatial derivatives (gradients of reconstructed surface); normals $\rightarrow$ depth via integration (a PDE/reconstruction problem).

Angular Error for Normals

$$\theta = \arccos\left(\frac{\boldsymbol{n} \cdot \boldsymbol{n}^*}{\|\boldsymbol{n}\| \, \|\boldsymbol{n}^*\|}\right).$$

A common loss uses the dot product (cosine similarity) per pixel.

10.4 3D Shape Representations

10.4.1 Voxel Grids

Voxel Grid

Discretize 3D space into a $V \times V \times V$ grid; each voxel stores occupancy (or related properties). Conceptually "segmentation in 3D".

- (+) Simple, regular structure; can use 3D CNNs.
- (–) Memory scales as $O(V^3)$; high resolution is expensive.

Example learning pipeline: 2D CNN features $\rightarrow$ 3D CNN $\rightarrow$ voxel occupancy; trained with per-voxel cross-entropy loss.

10.4.2 Point Clouds

Point Cloud

A **point cloud** represents a shape as a set of P 3D points, typically on the surface.

- (+) Efficient for fine structures without a dense 3D grid.
- (–) Unordered set $\Rightarrow$ needs special architectures/losses; no explicit connectivity or surface.
- (–) Rendering or surface extraction often needs post-processing (e.g., meshing).

ICP (Iterative Closest Point) alternates between nearest-neighbor matching and estimating a rigid transform. Naive correspondence search is $O(N^2)$; with a k-d tree it is roughly $O(N \log N)$ per iteration.

10.4.3 Meshes

Mesh

Mesh: vertices + faces (usually triangles). Standard representation in graphics; explicitly encodes surface connectivity.

- (+) Adaptive: allocate more triangles where detail is high; can attach attributes (RGB, texture, normals).
- (–) Learning/optimization is harder than voxels due to topology/connectivity constraints.

10.4.4 Implicit Surfaces

Implicit Surface

Implicit surface: represent geometry via a function $f(x, y, z)$ (e.g., occupancy / inside-outside / density), rather than an explicit list of points or voxels. Connects naturally to rendering and NeRF-style methods.

10.5 Neural Radiance Fields (NeRF)

10.5.1 Camera Rays and Volume Rendering

A pixel corresponds to a **camera ray**

$$\boldsymbol{r}(t) = \boldsymbol{o} + t\,\boldsymbol{d},$$

with ray origin $\boldsymbol{o}$ at the camera center and direction $\boldsymbol{d}$ through the pixel (after applying intrinsics/extrinsics).

Volume Rendering Equation

The final color C of a pixel is the integral of light emitted along that ray, attenuated by transmittance:

$$C(\text{ray}) = \int_{\text{near}}^{\text{far}} T(t)\,\sigma(t)\,c(t)\,dt$$

- **Color (c):** the color emitted by a specific 3D point.
- **Density (σ):** how solid/opaque is the point?
- **Transmittance (T):** probability that light from this point reaches the camera without obstruction.

10.5.2 NeRF Parameterization and Training

NeRF

Train an MLP that maps position $\boldsymbol{p} = (x, y, z)$ and viewing direction $\boldsymbol{d} = (\theta, \phi)$ to

$$(\sigma(\boldsymbol{p}),\ \boldsymbol{c}(\boldsymbol{p}, \boldsymbol{d})).$$

Rendering an image is then differentiable volume rendering along camera rays.

Training: given multiple **posed images** (known camera intrinsics/extrinsics), sample rays and minimize an image reconstruction loss between rendered pixel colors and observed pixel colors (often L_2).

Key limitations:

- **Scene-specific:** classical NeRFs are trained per scene (no universal pretraining by default).
- **Compute:** training and rendering are expensive due to many samples along many rays.
- **Challenges:** dynamic scenes, strong view-dependent effects, and occlusions/thin structures.

If σ (opacity/density) were forced to be **binary** (0/1), the model can still represent **opaque surfaces** (diffuse and view-dependent/specular appearance), but cannot represent **transmission/refraction** or semi-transparent volumetric effects.

NeRF is primarily about **novel view synthesis** (rendering from new viewpoints), but it also induces a rich 3D scene representation (density field) that can be queried for geometry and depth cues.